Infrastructure Risk Analyzer

Technical Guide — Deployment & Configuration Reference

For Data Scientists and IT Teams
World Bank Digital Development
November 2026 | Version 2.1

Created by Sebastian Krantz

Official Use Only

1. Overview

What is the Infrastructure Risk Analyzer?

The Infrastructure Risk Analyzer is a full-stack web application that automates climate and natural-hazard risk analysis for digital infrastructure assets. Users upload spatial infrastructure datasets (points or lines), select hazard layers (flood, earthquake, cyclone, landslide, drought), and the tool calculates exposure statistics, maps affected assets, and optionally estimates economic damage using vulnerability curves.

It was developed at the World Bank for task teams working on infrastructure investment operations and climate risk disclosure (e.g., for Project Appraisal Documents).

Who Should Read This Document

This guide is for engineers, data scientists, or IT staff responsible for deploying, configuring, or maintaining the application in a new environment. It covers installation, configuration, and two deployment paths: a standard full-stack install and Posit Connect. For end-user instructions on how to use the tool, see the companion document *Infra Risk Analyzer — User Guide*.

Key Capabilities

- Upload infrastructure data in Shapefile, GeoPackage, GeoJSON, or CSV format (points and lines; polygons are converted to centroids)
- Visualize infrastructure against 29 hazard layers from the GIRI platform (floods, tropical cyclones, earthquakes, landslides, drought, plus building-value and population context layers) on an interactive map
- Adjust exposure thresholds and view affected vs. unaffected asset counts or lengths
- Optionally estimate economic damage using vulnerability curves (with optional uncertainty bounds) and replacement costs
- Export maps and bar charts as high-resolution PNG images, and analysis results as CSV or GeoPackage

2. How It Works

System Architecture

The application is a two-tier web application:

- **Backend:** A Python FastAPI REST API that handles file uploads, raster sampling, tile rendering, and export.
- **Frontend:** A React single-page application (SPA) that provides an interactive map and control sidebar, running entirely in the user's browser.
- **Data layer:** Hazard layers are Cloud-Optimized GeoTIFFs (COGs), accessed either from remote URLs or from local disk. A semicolon-delimited CSV file (hazard_layers.csv) acts as the registry of available hazard layers.

Data Flow

- The user uploads an infrastructure file → the backend parses it, reprojects all coordinates to WGS84, stores the GeoDataFrame in process memory, and returns the dataset as GeoJSON for display.
- The frontend requests hazard tile images (/api/tiles/{id}/{z}/{x}/{y}.png) — the backend reads the relevant window from the COG raster, renders it with a matplotlib colormap, and caches the result (5-minute TTL, up to 2,000 tiles).
- The user triggers analysis → the backend samples raster values along each asset (lines are sampled at every vertex and at most 100 m apart, and split into affected and unaffected segments), classifies assets against the threshold, and returns a GeoJSON feature collection plus summary statistics. Raster values are cached per file × hazard combination (the eight most recent), so threshold changes do not re-read the raster.
- Export endpoints render high-resolution PNG images of maps and bar charts using matplotlib and contextily, and write the analysis results as CSV or GeoPackage.

Official Use Only

- On the Posit Connect build (branch posit), FastAPI also serves the compiled frontend (index.html and /assets) and returns index.html for any other GET path, so the whole application runs as a single process.

Critical Design Constraint

> **Important:** All uploaded files and analysis results are stored in Python process memory. The backend must always run with a single worker (--workers 1). Running multiple workers without a shared cache (such as Redis) causes each worker to have a different view of uploaded files, leading to silent failures.

3. Software Stack

Backend (Python)

Component	Library	Minimum Version
API framework	FastAPI + Uvicorn	≥ 0.104.1 / ≥ 0.24.0
File uploads	python-multipart	≥ 0.0.6
Vector I/O	geopandas + pyogrio	≥ 0.14.0 / ≥ 0.5.0
Raster I/O	rasterio	≥ 1.3.9
Raster arrays	xarray + rioxarray	≥ 2023.10.0 / ≥ 0.16.0
Data processing	pandas + numpy	≥ 2.1.0 / ≥ 1.24.0
Image/tile rendering	Pillow + matplotlib + seaborn	≥ 10.1.0 / ≥ 3.8.0 / ≥ 0.13.0
Map export	contextily	≥ 1.4.0
Caching	cachetools	≥ 5.3.0
Settings file	python-dotenv	any

Python version: **3.11** is recommended: it is what the Posit Connect build pins (3.11.9, in backend/manifest.json and .python-version), and the app also runs on 3.13. Python 3.10 or newer is required. requirements.txt gives minimum versions only (the same on both branches); pin exact versions with pip freeze for reproducible production installs. The rasterio and pyogrio wheels bundle GDAL, so no system GDAL is needed for a pip install.

Frontend (Node.js)

Component	Library	Version
UI framework	React + TypeScript	18.2.0 / 5.2.2
Build tool	Vite	5.0.8
Map rendering	MapLibre GL	3.6.2
Geospatial layers	deck.gl	9.0.0
Charts	Recharts	2.10.3
Styling	TailwindCSS	3.4.0

Official Use Only

Node.js 18 or higher is required to build the frontend (20 LTS recommended). Versions above are the minimum ranges in package.json; package-lock.json pins the exact versions.

4. System Requirements

Requirement	Minimum	Recommended
RAM	4 GB	8–16 GB
CPU	2 cores	4+ cores
Disk (raster data)	Minimal with remote layers	About 50 GB free to store all 29 layers locally
Operating system	Linux or macOS	Ubuntu 22.04 or 24.04 LTS
Python	3.10	3.11
Node.js (build only)	18	20 LTS
Network	Outbound HTTPS	See the note below

Note on network access: The server needs outbound HTTPS to hazards-data.unepgrid.ch when hazard layers are read remotely (only the tile windows needed are fetched), and to the basemap tile hosts when exporting maps: basemaps.cartocdn.com, tile.openstreetmap.org, a.tile.opentopomap.org, server.arcgisonline.com and services.arcgisonline.com. Users' browsers load basemap tiles directly from the same hosts, and from b. and c.tile.opentopomap.org, so a proxy or content-security policy in front of users must allow them too.

5. Repository Structure

```
InfraRiskAnalyzer/                (branch main)
├── README.md                     # Overview and quick start
├── docs/                         # This guide (Word and PDF)
├── data/
│   ├── hazard_layers.csv         # Hazard layer registry (semicolon-delimited)
│   └── rasters/                  # Optional local GeoTIFFs (not in git)
├── scripts/
│   └── download_hazard_rasters.py    # Download all layers, rewrite the CSV to local paths
├── backend/
│   ├── .env.example              # CARTO_API_KEY; copy to backend/.env
│   ├── main.py                   # FastAPI entry point
│   ├── requirements.txt          # Python dependencies
│   └── app/
│       ├── config.py             # Paths and environment variables
│       ├── api/                  # upload, hazards, analyze, tiles, export
│       └── utils/                # geospatial.py (sampling, curves), export_data.py
└── frontend/
    ├── package.json, vite.config.ts
    ├── public/infra-risk-analyzer-guide.pdf   # User guide opened by the Guide button
    └── src/                      # App.tsx, components/, services/api.ts
```


Official Use Only

The repository has two long-lived branches. Changes are made on main and merged into posit, which adds the packaging for Posit Connect:

Branch	Use and additions	Frontend served by
main	Development; Option A (Section 7)	Separate web server (Vite in development, nginx or similar in production)
posit	Option B, Posit Connect (Section 8). Adds backend/static/ (compiled frontend, committed), backend/data/hazard_layers.csv (paths on the Posit server), backend/manifest.json and .python-version	FastAPI

6. Configuration Reference

Environment Variables

Variable	Default	Description
HOST	0.0.0.0	Server bind address
PORT	8000	Server port
DEBUG	False	Auto-reload when started with python main.py (development only)
UPLOAD_DIR	<backend>/uploads (Posit)	Created at start-up but not written to: an upload is parsed in the system temporary directory and then held in memory only. Posit build only; fixed to <repo>/uploads on main
DATA_DIR	<backend>/data (Posit)	Directory holding the hazard CSV and local rasters. Posit build only; fixed to <repo>/data on main
HAZARD_LAYERS_CSV	<DATA_DIR>/hazard_layers.csv	Explicit path to the hazard CSV; overrides DATA_DIR. Posit build only
CARTO_API_KEY	(empty)	CARTO requires an API key for its raster basemaps (Positron, the default, and Dark Matter); without it those tiles carry an "API KEY REQUIRED" watermark. Read from the environment or from backend/.env (copy backend/.env.example); a variable set in the environment wins. Used for map exports and by the Vite development server; the Posit build writes it into the page it serves, and a production build of main needs it written into dist/index.html (Section 7, Step 5b)
UPLOAD_TTL_HOURS	4	Hours an upload may go unused before it is deleted from memory, with its cached results; checked on each upload and every 10 minutes
CORS_ORIGINS	(empty)	Comma-separated origins of other sites allowed to call the API from a browser. Empty: none, as the app's own pages are same-origin
GDAL_*	(unset)	GDAL_DISABLE_READDIR_ON_OPEN=EMPTY_DIR, CPL_VSIL_CURL_ALLOWED_EXTENSIONS=.tif,.tiff,.vrt, GDAL_HTTP_MERGE_CONSECUTIVE_RANGES=YES, GDAL_HTTP_MULTIPLEX=YES, GDAL_CACHEMAX=512. Recommended for any deployment that reads remote COGs; the app sets none of them

Official Use Only

Tip: HAZARD_LAYERS_CSV is the most important variable for custom or server deployments. Set it when the CSV is not co-located with the deployed application code (e.g. on Posit Connect).

Hazard Layers CSV Format

The file hazard_layers.csv uses a semicolon (;) delimiter. It is read on the first request and cached for the life of the process, so restart the application after editing it. If the file is missing, the application still starts but offers no hazard layers.

Column	Required	Description
hazard	Yes	Display name shown in the UI hazard dropdown
dataset_url	Yes	Absolute filesystem path or HTTPS URL of a Cloud-Optimized GeoTIFF in EPSG:4326
description	No	Methodology or source description shown in the info popup
background_paper	No	Link to a background paper or methodology reference
unit	No	Intensity unit displayed in the UI (e.g. mm, km/h, cm/s²)
category	No	Group heading in the hazard dropdown; consecutive rows with the same category form one group

Example CSV content:

```
hazard;dataset_url;description;background_paper;unit;category
Flood Hazard 100 Years - Existing climate;https://hazards-data.unepgrid.ch/global_pc_h100glob.tif;...;...;mm;Flood hazard
Tropical Cyclone Wind - 100 Years;https://hazards-data.unepgrid.ch/Wind_T100.tif;...;...;km/h;Tropical cyclone
Peak Ground Acceleration PGA - 475 Years;/srv/infrarisk/rasters/PGA_475y.tif;...;...;cm/s²;Earthquakes
```

The layer ID used in API paths is derived from the hazard name, so renaming a layer changes its ID.

7. Deployment Option A — Standard Full-Stack

Use this approach for development, testing, or production deployment on a Linux or macOS server where you have direct shell access. It uses the main branch.

Prerequisites

- Python 3.10 or higher (3.11 recommended)
- Node.js 18 or higher (20 LTS recommended)
- Git

Step 1 — Clone the repository

```
git clone https://github.com/SebKrantz/InfraRiskAnalyzer.git
cd InfraRiskAnalyzer
```

Step 2 — Backend: create virtual environment and install dependencies

```
cd backend
python -m venv venv
source venv/bin/activate      # Windows: venv\Scripts\activate
```


Official Use Only

```
pip install -r requirements.txt
cp .env.example .env                # then set CARTO_API_KEY in it (Section 6)
```

Step 3 — Configure hazard layers

The registry data/hazard_layers.csv (at the repository root) ships with all 29 GIRI layers as remote URLs and works as is if the server has outbound HTTPS. To serve local copies instead, run python3 scripts/download_hazard_rasters.py from the repository root: it downloads every layer into data/rasters/, resumes interrupted downloads, backs up the original CSV to data/hazard_layers.remote.csv and rewrites each dataset_url to the local path (see Section 9).

Step 4 — Start the backend server

For development (with auto-reload on code changes):

```
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

For production (single worker — required):

```
uvicorn main:app --host 0.0.0.0 --port 8000 --workers 1
```

The REST API and interactive documentation will be available at: http://localhost:8000 and http://localhost:8000/docs.

Step 5a — Frontend: development mode

```
cd ../frontend
npm install
npm run dev
```

The application will be available at http://localhost:5173. The Vite development server automatically proxies all /api/* requests to the backend on port 8000; set BACKEND_URL to point it at another address.

Step 5b — Frontend: production build

```
cd frontend
npm run build
# Output: frontend/dist/
sed -i 's/"__CARTO_API_KEY__"/"<your-key>"/' dist/index.html   # CARTO key, see below
```

Serve the frontend/dist/ directory with any static file server. The server must proxy /api/* to the backend and return index.html as the fallback for all non-API routes (SPA routing). The built page carries a placeholder for the CARTO key, which the backend on main cannot fill in because it does not serve the page: write the key into dist/index.html after every build, as above, or the CARTO basemaps show an "API KEY REQUIRED" watermark. Browsers request the tiles with the key, so users can see it on either branch.

Nginx configuration example

```
server {
    listen 80;
    root /path/to/InfraRiskAnalyzer/frontend/dist;
    index index.html;
    client_max_body_size 100m;   # nginx allows 1 MB by default, too small for uploads

    location /api/ {
        proxy_pass http://localhost:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_read_timeout 300s;     # large analyses and map exports can take minutes
    }

    location / {
```


Official Use Only

```
        try_files $uri $uri/ /index.html;  # SPA fallback
    }
}
```

Important: Always use --workers 1. The app holds uploaded files and analysis results in Python process memory. Running multiple workers causes each process to have its own independent memory, so file uploads made to one worker are invisible to others.

8. Deployment Option B — Posit Connect

The posit branch adapts the application for Posit Connect. On this branch, the React frontend is compiled into backend/static/ and served directly by FastAPI, so only the Python app needs to be deployed and no Node.js process runs on the server. The compiled frontend is committed, so rebuild it (Step 2) after merging frontend changes from main.

Prerequisites

- Python 3.11.9 (must match Posit Connect's runtime; check your server's configured version)
- Node.js 18+ locally (for the one-time frontend build step, done on your machine before deploying)
- rsconnect-python CLI: pip install rsconnect-python
- A saved Posit Connect server configuration: rsconnect add --name <server-name> --server https://<posit-url> --api-key <key>

Step 1 — Switch to the Posit Connect branch

```
git checkout posit
```

Step 2 — Build the frontend into the backend directory

```
cd frontend
npm install
npm run build:connect    # Compiles to ../backend/static/
```

This compiles the React app into backend/static/, which FastAPI on this branch serves as static files, with a fallback route that returns index.html for all other paths (SPA routing). The build empties backend/static/ first and copies frontend/public/, including infra-risk-analyzer-guide.pdf, the user guide opened by the Guide button; replace that PDF before building to update the guide.

Step 3 — Deploy to Posit Connect

```
cd ..  # repo root
rsconnect deploy fastapi \
  -n <saved-server-name> \
  --entrypoint main:app \
  --exclude .env \
  backend/
```

--exclude .env keeps a local backend/.env out of the bundle; rsconnect would otherwise upload it. Posit Connect installs the packages in backend/requirements.txt into a Python 3.11 environment. The committed backend/manifest.json (entrypoint main:app, Python 3.11.9) records a file list with checksums; if you redeploy from it with rsconnect deploy manifest backend/manifest.json, first regenerate it with rsconnect write-manifest fastapi --overwrite --entrypoint main:app --exclude .env backend/ so that it matches the current files.

Step 4 — Set environment variables on Posit Connect

In the Posit Connect web interface, navigate to the deployed app's settings and set:

Official Use Only

Variable	Example Value	Notes
HAZARD_LAYERS_CSV	/srv/data/infrarisk/hazard_layers.csv	Absolute path to the CSV file on the Posit server
DATA_DIR	*(leave unset)*	Only set if both the CSV and TIFFs are in the same directory
CARTO_API_KEY	your CARTO key	Written into the page when it is served; without it the CARTO basemaps show a watermark

> **Important:** Do not set DATA_DIR to the TIFF directory unless hazard_layers.csv is also stored there. Use HAZARD_LAYERS_CSV to point directly at the CSV, regardless of where TIFFs live.

Step 5 — Set the runtime settings to a single process

In the content's **Settings → Runtime** panel, set **Max processes** to **1**. Posit Connect can run several processes for one content item and spreads requests across them, which splits the in-memory state: an upload lands in one process and the following analysis request in another, which answers "File not found". Set **Min processes** to **1** as well, so that Connect does not shut the app down when it is idle; a shutdown clears all uploads. Use **Settings → Access** to control who can open the app.

> **Important:** Max processes = 1 is the Posit equivalent of --workers 1 and is required for the app to work reliably.

Step 6 — Verify TIFF file permissions

The FastAPI process on Posit Connect runs as a specific OS user (e.g. shiny or rstudio-connect). All TIFF files listed in dataset_url must be readable by that user:

```
chmod o+r /path/to/rasters/*.tif
# Or, using group-based access:
chgrp rstudio-connect /path/to/rasters/*.tif
chmod g+r /path/to/rasters/*.tif
```

Uploads blocked by a web application firewall

If the Connect server sits behind a web application firewall, as datanalytics.worldbank.org does behind Cloudflare, uploads of GeoPackage files can be rejected before they reach the app. A GeoPackage is a SQLite database, and its bytes contain the SQL statements of its schema (CREATE TABLE, CREATE TRIGGER, SELECT, UPDATE), which SQL-injection rules match.

Symptoms: the app shows "Upload failed"; the browser's network tab shows HTTP 403 with an HTML body, server: cloudflare and a cf-ray ID; the app log has no entry for the request. **Fix:** in the firewall's security events, look up the cf-ray ID to find the rule that fired, and exempt POST /infra-risk-analyzer/api/upload from it. Until then, users can upload the same data as GeoJSON or as a zipped shapefile.

Troubleshooting

Symptom	Likely Cause	Fix
No hazard layers appear in the dropdown	CSV file not found by the app	Set the HAZARD_LAYERS_CSV environment variable to the absolute path of the CSV
Hazard layer listed but tiles or analysis fail	TIFF file is not readable	Check file permissions; inspect app logs for “Permission denied” errors
DATA_DIR is set but CSV still not found	CSV is not in the TIFF directory	Use HAZARD_LAYERS_CSV instead of DATA_DIR
500 errors on tile requests	Corrupt or incompatible TIFF	The app shows the real error (e.g. Permission denied, No such file) in the UI and the log; fix the path in the CSV or the file permissions

Official Use Only

Symptom	Likely Cause	Fix
Map export is slow or fails	The server cannot reach the basemap tile host	Allow outbound HTTPS to the tile hosts (Section 4); if tiles cannot be fetched, the export is produced without a basemap and a warning is logged
"Upload failed" for GeoPackage files; HTTP 403 from cloudflare	Firewall rule matches SQL text inside the GeoPackage	Exempt the upload route from the rule (see above)
"File not found" or results that vanish between requests	More than one process serves the content	Set Max processes to 1 (Step 5)
Uploads lost after a period of inactivity	Connect shut down the idle process	Set Min processes to 1 (Step 5)
Basemap tiles show "API KEY REQUIRED"	CARTO_API_KEY is not set	Set CARTO_API_KEY under Vars (Step 4) and restart the content

9. Hazard Data Configuration

Using Remote Cloud-Optimized GeoTIFFs

Set dataset_url to a public HTTPS URL. The app uses rasterio's virtual filesystem (VSICURL) to fetch only the spatial windows needed for each tile or analysis request; it does not download the full file. Outbound HTTPS access to the hosting domain is required, and the GDAL settings listed in Section 6 make remote reads markedly faster.

Using Local TIFFs

Set dataset_url to an absolute filesystem path (e.g. /data/rasters/flood_100yr.tif); scripts/download_hazard_rasters.py does this for the whole catalogue. For efficient tile serving, files should be Cloud-Optimized GeoTIFFs (COG) with internal overviews, in EPSG:4326. Convert or reproject with GDAL:

```
gdal_translate input.tif output_cog.tif -of COG -co COMPRESS=LZW                # already in EPSG:4326
gdalwarp -t_srs EPSG:4326 -of COG -co COMPRESS=DEFLATE input.tif output_cog.tif   # any other CRS
```

Non-COG files will still work but may be slower for tile requests.

Adding a New Hazard Layer

Add a new row to hazard_layers.csv and restart the application: the CSV is read once and cached for the life of the process. The new layer appears in the dropdown once users reload the page. Give it a category matching an existing group and place the row next to that group's other rows, because the dropdown groups consecutive rows.

Coordinate Reference Systems

Hazard rasters must be in WGS84 (EPSG:4326). The tile renderer can display a projected raster, but the analysis converts asset coordinates, which are always WGS84, directly into pixel positions, so a projected raster would give wrong exposure results. Reproject with gdalwarp as shown above. Uploaded infrastructure data is reprojected to WGS84 automatically.

10. Infrastructure Data Formats

Format	File Extension	Notes
Shapefile	.zip	ZIP archive must contain .shp, .shx, and .dbf files; include .prj for non-WGS84 data
GeoPackage	.gpkg	Single-layer files recommended; first layer is used if multiple exist
GeoJSON	.geojson	Standard GeoJSON format; WGS84 assumed

Official Use Only

Format	File Extension	Notes
CSV	.csv	Coordinate columns lat/lon, lat/lng, latitude/longitude or y/x (points), or a WKT column named geometry, geom or wkt (points or lines)

Supported geometry types: **Point** (telecom towers, substations, data centers) and **LineString** or **MultiLineString** (fiber cables, roads, pipelines), which may be mixed within a line layer. Polygon and MultiPolygon features are converted to centroid points. Points and lines cannot be mixed in one file. All data is reprojected to WGS84 on upload.

Lines are measured using geodesic calculations (meters, accounting for Earth's curvature) via pyproj's Geod class. This ensures accurate length statistics regardless of the original projection.

Maximum upload file size: **100 MB** (MAX_UPLOAD_SIZE in backend/app/config.py). Reverse proxies and firewalls in front of the app must allow request bodies of this size.

11. API Reference

All endpoints except /health are prefixed with /api. A full interactive API reference (Swagger UI) is available at http://<server>:8000/docs when the backend is running. On the Posit build, unknown GET paths return the web page with HTTP 200 rather than 404, so monitor /health.

Method	Endpoint	Description
POST	/api/upload	Upload an infrastructure file (multipart field file); returns a file_id, metadata and the dataset as GeoJSON
GET	/api/upload/{file_id}	Metadata of an uploaded dataset: file name, geometry type, feature count, CRS and bounds
DELETE	/api/upload/{file_id}	Remove an uploaded dataset and its cached results (Clear Data, a new upload, or leaving the page)
GET	/api/hazards	List all configured hazard layers
GET	/api/hazards/{hazard_id}	Details of one hazard layer
GET	/api/hazards/{hazard_id}/stats	Raster min/max used to configure the threshold slider
POST	/api/analyze	Exposure analysis, and damage estimation when a curve and replacement value are sent; raster values cached per file × hazard
GET	/api/analyze/{file_id}/status	Whether a dataset is loaded and ready
GET	/api/tiles/{hazard_id}/{z}/{x}/{y}.png	XYZ map tile of a hazard layer (MapLibre tile source)
POST	/api/tiles/clear-cache	Empty the tile cache
POST	/api/export/barchart	Bar chart as a high-resolution PNG
POST	/api/export/map	Map as a high-resolution PNG, with basemap
POST	/api/export/data	Results as CSV (csv_points, csv_lines_aggregate) or GeoPackage (gpkg_lines_split)
GET	/health	Health check; returns {"status": "healthy"}

Official Use Only

12. Known Constraints and Limitations

Single-Worker Architecture

All uploaded files and analysis cache are held in Python process memory. The backend must always run with `--workers 1`, and the platform must run a single instance: one process on Posit Connect (Max processes = 1). Adding multi-worker support would require replacing the in-memory stores with a shared external cache such as Redis or Memcached.

No Authentication

The application has no user accounts or access control. Uploaded datasets are held in one shared process and addressed by random IDs; there is no endpoint that lists them, but anyone who can reach the URL can use the app, including the unauthenticated `POST /api/tiles/clear-cache`. Restrict access at the platform or network level: Posit Connect access settings, a VPN, or an authenticating reverse proxy.

No Persistence

Uploaded files exist only in memory while the backend process is running; there is no database or persistent storage, and restarting the process clears all uploads and analysis caches. An upload is deleted, with its cached results, when the user clicks Clear Data, replaces it with another file or leaves the page, and otherwise after `UPLOAD_TTL_HOURS` (4 by default) without use, so memory does not grow with the number of uploads on a long-running server. The analysis caches keep the eight most recent file × hazard combinations.

Tile Cache Memory

Rendered tile images are cached in memory with a 5-minute TTL and a maximum of 2,000 tiles. Memory usage grows with active usage and is automatically managed by LRU eviction.

Basemaps and Third-Party Services

Basemap tiles are loaded from third-party services (CARTO, OpenStreetMap, OpenTopoMap, Esri) by users' browsers, and by the server for map exports. CARTO requires an API key for its raster basemaps, set with `CARTO_API_KEY` (Section 6); without it the CARTO options, including the default Positron basemap, show an "API KEY REQUIRED" watermark. Check each provider's terms of use for your deployment.

Raster Data Size

Large rasters (multi-GB TIFFs) are handled efficiently via COG windowed reads — only the spatial windows needed for each tile or sample are fetched. However, analysis of very large infrastructure datasets (tens of thousands of features) may be slow for the initial sampling pass, which is then cached for subsequent threshold changes.

Reverse Proxies and Firewalls

Anything in front of the app must allow request bodies of up to 100 MB (uploads), read timeouts of several minutes (analysis and map export of large datasets), and GeoPackage uploads, which web application firewalls can mistake for SQL injection (Section 8). The upload response contains the whole dataset as GeoJSON; the Posit build compresses JSON responses with gzip, so keep compression enabled on proxies.

Official Use Only